

ECE 661 Homework 3

Student's Name: Syed Ali A. Shah
Student's Email shah634@purdue.edu

04 Sept. 2024

Introduction:

ECE661 Fall 2024: Homework 3
TA: Rahul Deshmukh (deshmuk5@purdue.edu)
Due Date: Midnight, 11 Sept 2024
Late submissions will be accepted with penalty: -10 points per-late-day, up to 5 days.

Contents

1	Solution to Task-1	2
1.1	Logic used for Task-1	2
1.1.1	Reasoning/Methodology for Task-1 Part 1 (Point-to-point correspondences)	2
1.1.2	Reasoning/Methodology for Task-1 Part 2 (Two-Step Method)	2
1.1.3	Reasoning/Methodology for Task-1 Part 3 (One-Step Method)	4
1.2	Figures for Task-1 and Its Parts	6
1.2.1	Figures for Task-1 Point to Point Correspondences:	7
1.2.2	Figures for Task-1 Two Step:	8
1.2.3	Figures for Task-1 One Step:	11
1.3	Observations of the different methods:	11
2	Solution to Task-2	12
2.1	Reasoning/Methodology for Task-2	12
2.2	Figures for Task-2 and Its Parts	13
2.2.1	Figures for Task-2 Two Step:	14
2.2.2	Figures for Task-2 One Step:	18
3	Python Code for Tasks	18

1 Solution to Task-1

1.1 Logic used for Task-1

1.1.1 Reasoning/Methodology for Task-1 Part 1 (Point-to-point correspondences)

To estimate the homography between a pair of images, we start with the understanding that the relationship between corresponding points is governed by a homography matrix H . The points in the domain image (undistorted) and the range image (distorted) are related by the equation:

$$\mathbf{x}' = \mathbf{H}\mathbf{x}$$

where $\mathbf{x}$ and $\mathbf{x}'$ are the homogeneous coordinates of points in the original and distorted images, respectively.

If $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$ and $\mathbf{x}' = \begin{pmatrix} x'_1 \\ x'_2 \\ x'_3 \end{pmatrix}$, and the homography matrix H is defined as:

$$\mathbf{H} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix}$$

then the following system of equations arises:

$$\begin{pmatrix} x'_1 \\ x'_2 \\ x'_3 \end{pmatrix} = \mathbf{H} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

This matrix multiplication expands into three separate equations, but since homogeneous coordinates represent ratios of values, we can simplify the system. Each pair of corresponding points between the original and distorted images gives rise to two equations:

$$xh_{11} + yh_{12} + h_{13} - xx'h_{31} - yy'h_{32} = x'h_{33}$$

$$xh_{21} + yh_{22} + h_{23} - xy'h_{31} - yy'h_{32} = y'h_{33}$$

where $x = \frac{x_1}{x_3}$, $y = \frac{x_2}{x_3}$, and similarly $x' = \frac{x'_1}{x'_3}$ and $y' = \frac{x'_2}{x'_3}$.

Since only the ratios matter in the homography matrix, we can simplify by setting $h_{33} = 1$. This leaves us with eight unknowns ($h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}$). To solve for these unknowns, we need at least four pairs of corresponding points, which provide the necessary eight equations.

The system of equations can be written in matrix form as:

$$\begin{pmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1x'_1 & -y_1x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -x_1y'_1 & -y_1y'_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x_2x'_2 & -y_2x'_2 \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -x_2y'_2 & -y_2y'_2 \\ x_3 & y_3 & 1 & 0 & 0 & 0 & -x_3x'_3 & -y_3x'_3 \\ 0 & 0 & 0 & x_3 & y_3 & 1 & -x_3y'_3 & -y_3y'_3 \\ x_4 & y_4 & 1 & 0 & 0 & 0 & -x_4x'_4 & -y_4x'_4 \\ 0 & 0 & 0 & x_4 & y_4 & 1 & -x_4y'_4 & -y_4y'_4 \end{pmatrix} \begin{pmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{pmatrix} = \begin{pmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ x'_3 \\ y'_3 \\ x'_4 \\ y'_4 \end{pmatrix}$$

By solving this system of equations, we can compute the eight unknowns and reconstruct the homography matrix H . The homography matrix can then be applied to the original image to remove projective distortions.

1.1.2 Reasoning/Methodology for Task-1 Part 2 (Two-Step Method)

In the two-step method, we first remove projective distortion and then affine distortion. Both of these distortions are corrected using a homography matrix.

Step 1: Remove Projective Distortion To remove projective distortion, we rely on vanishing points and vanishing lines. In an image with projective distortion, parallel lines in the original scene converge at a point in the image called the vanishing point. The vanishing point $\mathbf{v}$ can be computed by taking the cross product of two points on a line $\mathbf{l}$:

$$\mathbf{v} = \mathbf{p}_1 \times \mathbf{p}_2$$

where $\mathbf{p}_1$ and $\mathbf{p}_2$ are points on the line $\mathbf{l}$.

Once we have two vanishing points corresponding to two pairs of parallel lines in the scene, we can compute the vanishing line $\mathbf{l}_\infty$ by taking the cross product of the two vanishing points:

$$\mathbf{l}_\infty = \mathbf{v}_1 \times \mathbf{v}_2$$

In the projective distorted image, the vanishing line is not at infinity. To correct this distortion, we apply a homography that maps the vanishing line back to the line at infinity, represented as:

$$\mathbf{l}_\infty = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

If the vanishing line in the image is given by:

$$\mathbf{l} = \begin{pmatrix} l_1 \\ l_2 \\ l_3 \end{pmatrix}$$

then the homography that moves this line back to infinity is:

$$\mathbf{H}^{-T} = \begin{pmatrix} 1 & 0 & -\frac{l_1}{l_3} \\ 0 & 1 & -\frac{l_2}{l_3} \\ 0 & 0 & \frac{1}{l_3} \end{pmatrix}$$

Step 2: Remove Affine Distortion Once projective distortion is removed, the next step is to remove affine distortion. To do this, we need to restore the angles between orthogonal lines to 90° . This can be done by analyzing the relationship between the lines and the dual degenerate conic $\mathbf{C}_\infty^*$. The cosine of the angle between two lines $\mathbf{l}$ and $\mathbf{m}$ can be expressed as:

$$\cos(\theta) = \frac{\mathbf{l}^T \mathbf{C}_\infty^* \mathbf{m}}{\sqrt{(\mathbf{l}^T \mathbf{C}_\infty^* \mathbf{l})(\mathbf{m}^T \mathbf{C}_\infty^* \mathbf{m})}}$$

To remove affine distortion, we need to ensure that the angle between two lines is orthogonal, i.e., $\theta = 90^\circ$, so that $\cos(\theta) = 0$. This gives us the following constraint for the affine homography H :

$$\mathbf{l}'^T \mathbf{H} \mathbf{C}_\infty^* \mathbf{H}^T \mathbf{m}' = 0$$

where $\mathbf{l}'$ and $\mathbf{m}'$ are lines in the image.

Assuming the affine transformation matrix H is of the form:

$$H = \begin{pmatrix} A & \mathbf{t} \\ 0^T & 1 \end{pmatrix}$$

where A is a 2×2 matrix, we can solve for A by analyzing orthogonal line pairs. Let $S = AA^T = \begin{pmatrix} s_{11} & s_{12} \\ s_{12} & s_{22} \end{pmatrix}$.

By fixing $s_{22} = 1$, we are left with two unknowns, s_{11} and s_{12} .

The system of equations to solve for s_{11} and s_{12} becomes:

$$s_{11}l'_1m'_1 + s_{12}(l'_1m'_2 + l'_2m'_1) = -l'_2m'_2$$

where $\mathbf{l}' = \begin{pmatrix} l'_1 \\ l'_2 \\ 1 \end{pmatrix}$ and similarly for $\mathbf{m}'$. We need at least two orthogonal line pairs to solve this system.

Assume the lines are denoted as $\mathbf{p}$ and $\mathbf{q}$. The system of equations for the unknowns s_{11} and s_{12} can be written as:

$$\begin{pmatrix} l'_1 m'_1 & l'_1 m'_2 + l'_2 m'_1 \\ p'_1 q'_1 & p'_1 q'_2 + p'_2 q'_1 \end{pmatrix} \begin{pmatrix} s_{11} \\ s_{12} \end{pmatrix} = \begin{pmatrix} -l'_2 m'_2 \\ -p'_2 q'_2 \end{pmatrix}$$

With these two equations, we solve for s_{11} and s_{12} .

Next, we apply Singular Value Decomposition (SVD) to the affine matrix A :

$$A = VD^2V^T$$

Substituting into $S = AA^T$, we get:

$$S = VD^2V^T$$

where $D^2 = \begin{pmatrix} \lambda_1^2 & 0 \\ 0 & \lambda_2^2 \end{pmatrix}$ represents the squared singular values λ_1 and λ_2 .

Thus, A is computed as:

$$A = V \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} V^T$$

Finally, the affine homography matrix H is constructed as:

$$H = \begin{pmatrix} A & 0 \\ 0^T & 1 \end{pmatrix}$$

By applying this matrix, we remove the affine distortion and obtain the rectified image.

Once we have solved for A , we can construct the affine homography as:

$$H = \begin{pmatrix} A & 0 \\ 0^T & 1 \end{pmatrix}$$

Thus, using these two steps, we can first eliminate projective distortion and then correct the affine distortion to obtain a fully rectified image.

1.1.3 Reasoning/Methodology for Task-1 Part 3 (One-Step Method)

In the one-step method, we eliminate both projective and affine distortions by estimating the dual degenerate conic $\mathbf{C}'_\infty$ and applying a homography to map it back to the ideal conic.

The transformed dual degenerate conic can be written as:

$$\mathbf{C}'^* = \mathbf{H}\mathbf{C}'_\infty\mathbf{H}^T$$

We can express $\mathbf{C}'^*$ as:

$$\mathbf{C}'^* = \begin{pmatrix} a & b/2 & d/2 \\ b/2 & c & e/2 \\ d/2 & e/2 & f \end{pmatrix}$$

Expanding $\mathbf{H}\mathbf{C}'_\infty\mathbf{H}^T$ gives:

$$\mathbf{H}\mathbf{C}'_\infty\mathbf{H}^T = \begin{pmatrix} AA^T & A\mathbf{v} \\ \mathbf{v}^T A & \mathbf{v}^T \mathbf{v} \end{pmatrix}$$

which leads to the following relationships:

$$AA^T = \begin{pmatrix} a & b/2 \\ b/2 & c \end{pmatrix}, \quad A\mathbf{v} = \begin{pmatrix} d/2 \\ e/2 \end{pmatrix}$$

To estimate the conic $\mathbf{C}'^*$, we assume two orthogonal lines $\mathbf{l}$ and $\mathbf{m}$ in the image. These lines satisfy the condition:

$$\mathbf{l}'^T \mathbf{C}'^* \mathbf{m}' = 0$$

Substitute $\mathbf{l}' = \begin{pmatrix} l'_1 \\ l'_2 \\ 1 \end{pmatrix}$ and $\mathbf{m}' = \begin{pmatrix} m'_1 \\ m'_2 \\ 1 \end{pmatrix}$ into the equation:

$$(l'_1 \quad l'_2 \quad 1) \begin{pmatrix} a & b/2 & d/2 \\ b/2 & c & e/2 \\ d/2 & e/2 & 1 \end{pmatrix} \begin{pmatrix} m'_1 \\ m'_2 \\ 1 \end{pmatrix} = 0$$

Expanding this, we get:

$$l'_1 m'_1 a + (l'_1 m'_2 + l'_2 m'_1) \frac{b}{2} + l'_2 m'_2 c + l'_1 \frac{d}{2} + m'_1 \frac{d}{2} + l'_2 \frac{e}{2} + m'_2 \frac{e}{2} = -1$$

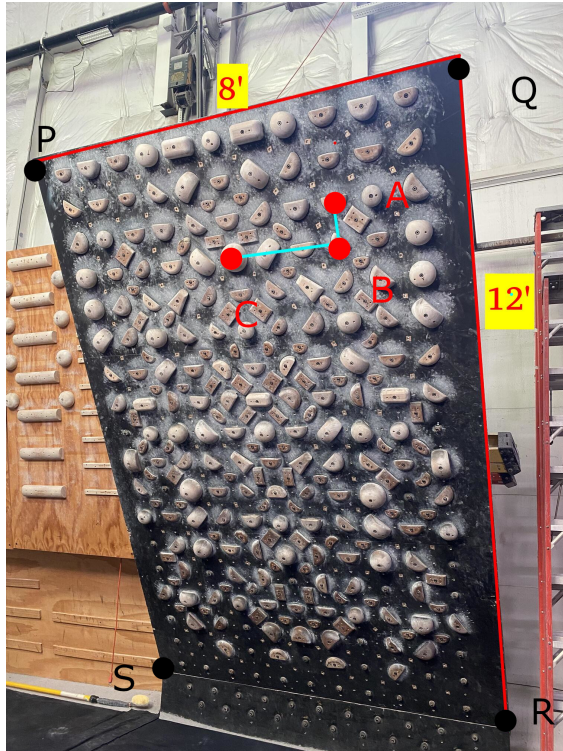
This equation contains five unknowns: a , b , c , d , and e . To solve for these variables, we need five pairs of orthogonal lines. Once we have estimated $\mathbf{C}'^*$, the relationship between A , $\mathbf{v}$, and $\mathbf{C}'^*$ can be used to construct the homography matrix H :

$$H = \begin{pmatrix} A & \mathbf{v} \\ 0^T & 1 \end{pmatrix}$$

By applying this homography, we can remove both projective and affine distortions and obtain the rectified image.

1.2 Figures for Task-1 and Its Parts

The figures and results of each part for Task-1 are given below.



(a) Board image



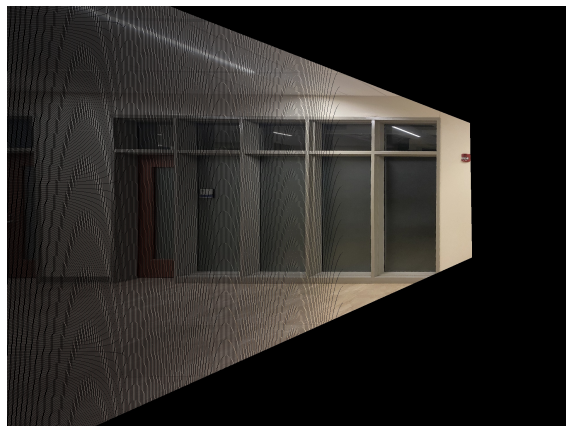
(b) Corridor image

Figure 1: Original Images with points for Board and Corridor images

1.2.1 Figures for Task-1 Point to Point Correspondences:



(a) Point to Point Correspondences results for Board image



(b) Point to Point Correspondences results for Corridor image

Figure 2: Results for Task-1 Point to Point Correspondences for Board and Corridor images

1.2.2 Figures for Task-1 Two Step:



(a) Results after removing Projective distortion for the Board image



(b) Results after removing Affine distortion for the Board image

Figure 3: Results of Task-1; Two Step for Board Images(Last result on next page)

0.45

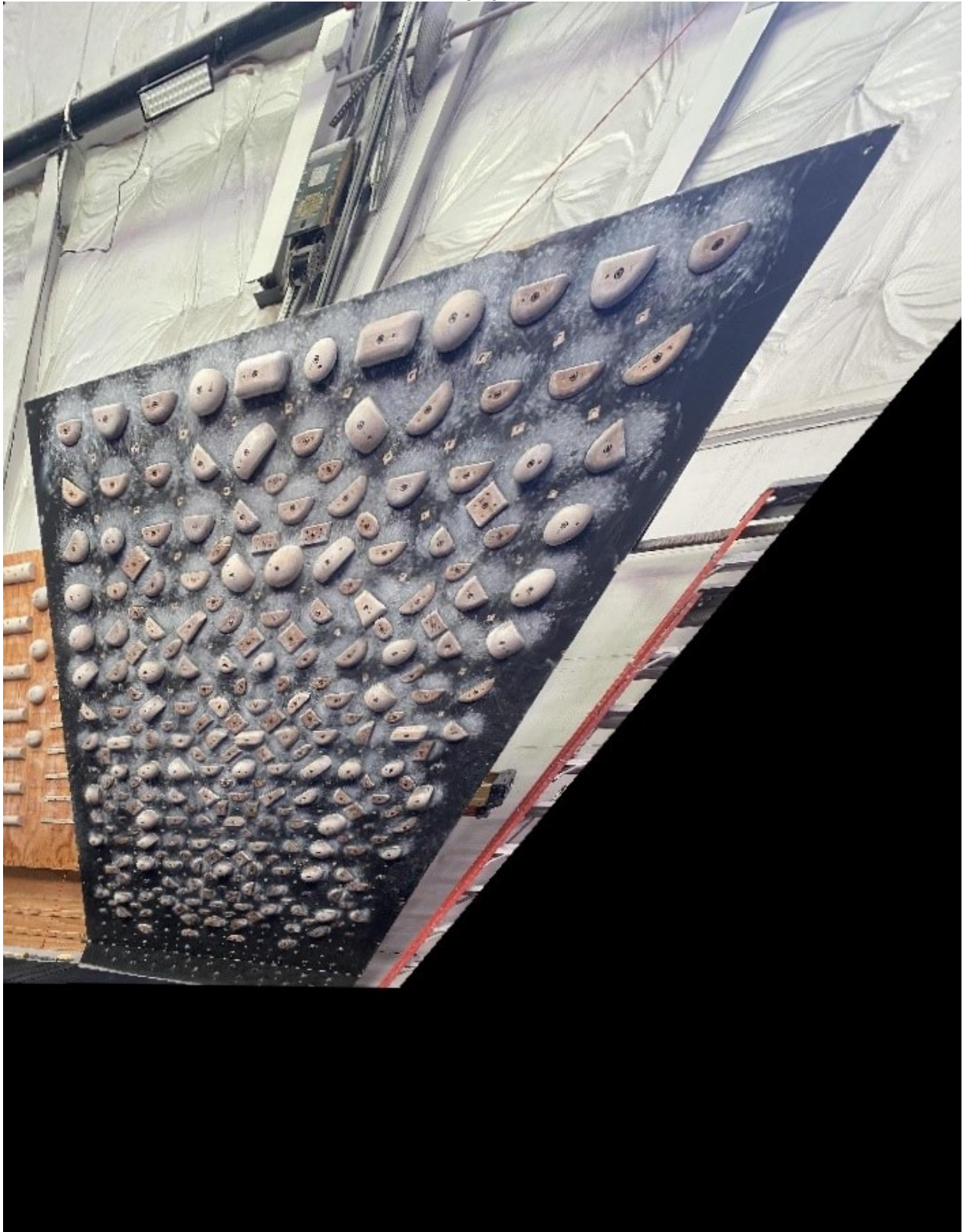


Figure 4: Results after removing both Projective and Affine distortion for the Board image



(a) Results after removing Projective distortion for the Corridor image



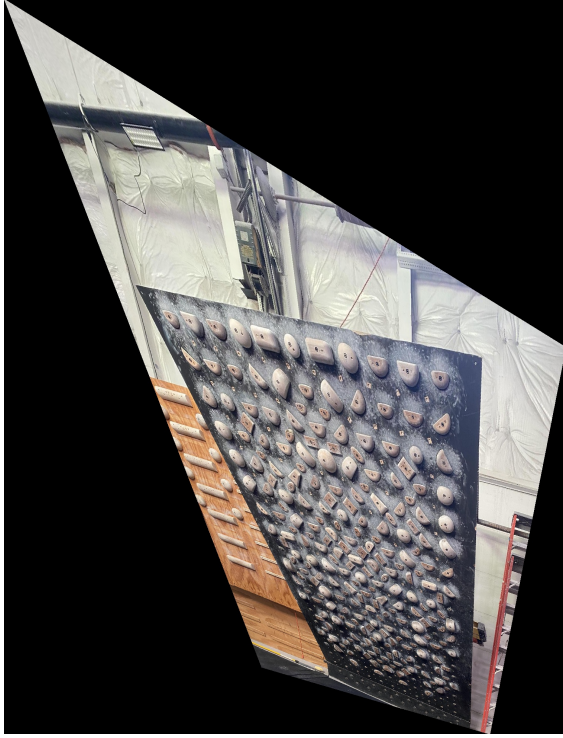
(b) Results after removing Affine distortion for the Corridor image



(c) Results after removing both Projective and Affine distortion for the Corridor image

Figure 5: Results of Task-1; Two Step for Corridor Images

1.2.3 Figures for Task-1 One Step:



(a) One Step results for Board image



(b) One Step results for Corridor image

Figure 6: Results of Task-1 One Step for Board and Corridor images

1.3 Observations of the different methods:

Point-to-Point The point-to-point method was applied to both the `board_1` and `corridor_1` images. The results were successful in rectifying the projective and affine distortions. My images, due to mapping issue, are blurry, however the region of interest is clearly, corrected with respect to the distortions. The calculated homographies mapped the distorted images to a more geometrically correct perspective. The transformation preserved the alignment of the key points, resulting in images where the overall structure appeared more natural and undistorted. However, this method relies heavily on accurate point selection and doesn't explicitly separate the projective and affine components of the distortion, which limits its flexibility.

Two Step: The two-step approach was applied to remove the projective distortion in the first step, followed by the removal of affine distortion in the second step. The first step, which focused on eliminating projective distortion using vanishing lines, resulted in a significant reduction of perspective distortion. The second step, involving affine correction, further rectified scaling and angular distortions. The transformations were highly effective. The key points aligned correctly, and the previously skewed structures in the `board_1` and `corridor_1` images appeared to be straightened. The parallel lines became parallel, and the orthogonal lines became perpendicular as expected. (Any issues seen in the images are due to scaling issues, and errors in calculating the bounds however the regions of interest clearly are fixed) After applying both steps, the affine correction further improved the image, creating a visually accurate and geometrically correct result. In the one-step approach, both projective and affine distortions were corrected simultaneously using a combined homography. The images generated by this method had minimal distortions, similar to the results obtained with the two-step approach. The straightness of the lines and the preservation of angles were prominent, and the overall quality was comparable to the results achieved using the two-step method. This is more effective at producing a geometrically accurate result.

One Step: The one-step method aimed tries to correct both projective and affine distortions simultaneously. However, being a simpler process, the one-step method produced results that were comparable to the two-step approach. The images showed minimal distortions, and both parallel and orthogonal lines appeared well-aligned. However, this method may not offer the same level of fine-tuning as the two-step process since it does not separate the two types of distortion, this is clearly shown by the fact the images, pull to one corner, if the points are not perfectly, chosen, this would work better with multiple points, representing the conics, via lines. (Any issues seen in the images are due to scaling issues, and errors in calculating the bounds however the regions of interest clearly are fixed)

Clearly, out of all methods, the two step approach yields the best results giving us better control over the homographies and rectifying the distortions.

2 Solution to Task-2

2.1 Reasoning/Methodology for Task-2

[Explanation of the approach for Task-2, similar to Task-1.]

2.2 Figures for Task-2 and Its Parts

The figures and results of Task-2 are provided below.



(a) Keyboard image (A, B, C is from the Tab key, to the letter Y, to the F6 key respectively)



(b) Door image(Measurements are 25inches by 9 inches for the region outlined by the points PQRS)

Figure 7: Original Images with points for Keyboard and Door images



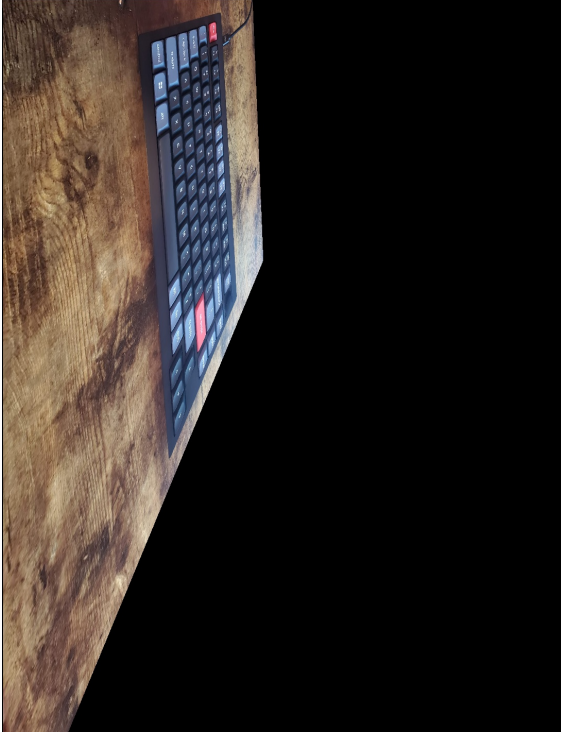
(a) Point to Point Correspondences results for Keyboard image



(b) Point to Point Correspondences results for Door image

Figure 8: Results for Task-2 Point to Point Correspondences for Keyboard and Door images

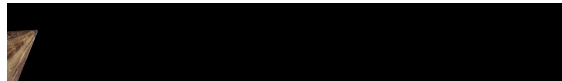
2.2.1 Figures for Task-2 Two Step:



(a) Results after removing Projective distortion for the Keyboard image

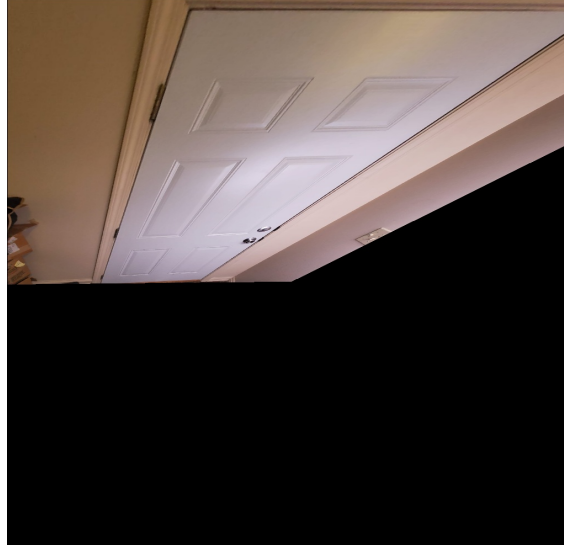


(b) Results after removing Affine distortion for the Keyboard image



(c) Results after removing both Projective and Affine distortion for the Keyboard image

Figure 9: Results of Task-2; Two Step for Keyboard Images



(a) Results after removing Projective distortion for the Door image

Figure 10: Results of Task-2; Two Step for Door Images(More results on next page)

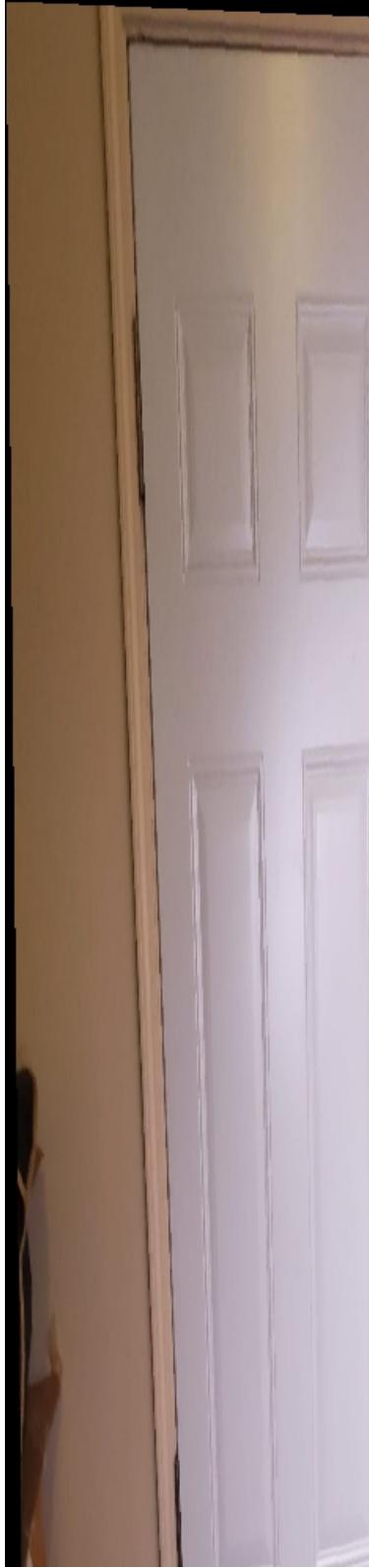


Figure 11: Results after removing Affine distortion for the Door image

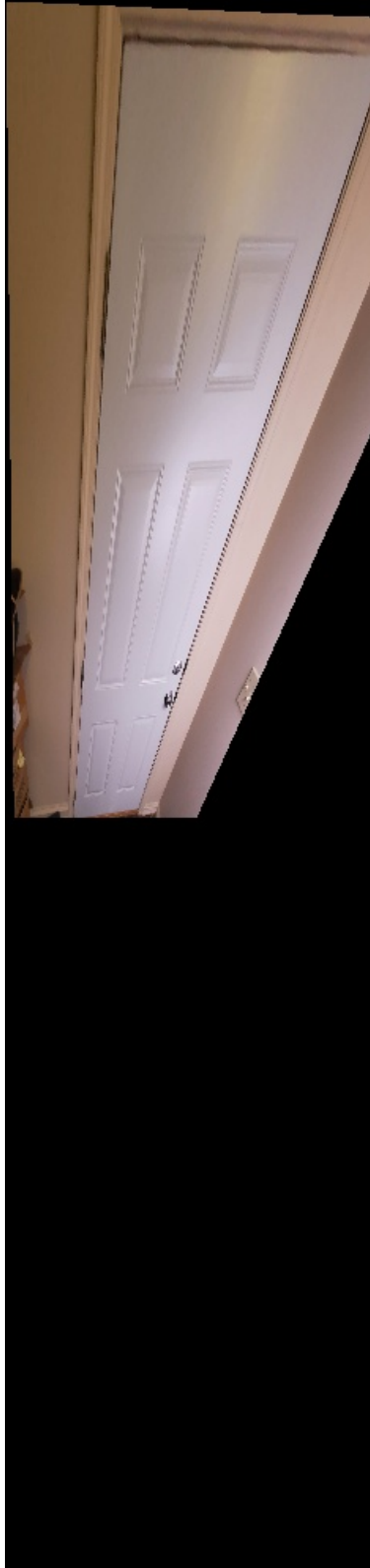


Figure 12: Results after removing both Projective and Affine distortion for the Door image

2.2.2 Figures for Task-2 One Step:



(a) One Step results for Keyboard image



(b) One Step results for Door image

Figure 13: Results of Task-2 One Step for Keyboard and Door images

3 Python Code for Tasks

```
1 import numpy as np
2 import cv2
3 from termcolor import cprint
4 from skimage import io
5 import matplotlib.pyplot as plt
6 from typing import List, Tuple
7
8 test = 0
9 H_STAR = None
10 def point_display(img, points):
11     plt.imshow(img)
12     plt.scatter(points[:, 0], points[:, 1], color='red')
13     plt.show()
14
15 def get_range_points(domain_pts: np.ndarray, ratio: int) -> np.ndarray:
16     P, Q, R, S = domain_pts
```

```

17     w= Q[0] - P[0]
18     h = int(ratio*w)
19     P_range = P
20     Q_range = [ Q[0], P[1] , 1]
21     R_range = [ Q[0], h+P[1] , 1]
22     S_range = [ P[0], h+P[1] , 1]
23     return np.array([ P_range, Q_range, R_range, S_range ] )
24
25 def calc_bounds(H: np.ndarray, points: np.ndarray) -> Tuple[int, int, int,
int]:
26     x_min, x_max, y_min, y_max = 0,0,0,0
27     for point in points:
28         temp = np.dot(H, np.array([point]).T)
29         temp = temp.flatten() # Flatten to make sure it's 1D for indexing
30         x_min, x_max = min(x_min, temp[0]), max(x_max, temp[0])
31         y_min, y_max = min(y_min, temp[1]), max(y_max, temp[1])
32     return int(x_min), int(x_max), int(y_min), int(y_max)
33
34 # Util function for printing out a formatted numpy array
35 def print_matrix(matrix: np.ndarray):
36     for row in matrix:
37         cprint(" ".join(f"{value:.3f}" for value in row), "white")
38
39 '''-----Task 1 Point to Point
-----'''
40 def calc_point_to_point_homography(src_pts: np.ndarray, dest_pts: np.ndarray)
-> np.ndarray:
41     A = np.zeros((8, 8))
42     b = np.zeros((8,))
43
44     for i in range(4):
45         x, y, _ = src_pts[i]
46         x_prime, y_prime, _ = dest_pts[i]
47         A[2 * i] = [x, y, 1, 0, 0, 0, -x * x_prime, -y * x_prime]
48         A[2 * i + 1] = [0, 0, 0, x, y, 1, -x * y_prime, -y * y_prime]
49         b[2 * i] = x_prime
50         b[2 * i + 1] = y_prime
51
52     print_matrix(A) # DEBUG STATEMENT!!!
53
54     try:
55         H_temp = np.dot(np.linalg.inv(A), b)
56         H = np.append(H_temp, 1).reshape((3, 3))
57     except np.linalg.LinAlgError:
58         cprint("Error: Matrix inversion failed. The matrix A might be
singular.", "red")
59         H = np.eye(3)
60     return H
61
62 def create_empty_canvas(width: int, height: int) -> np.ndarray:
63     limit = 50000 # Limit dimensions to prevent large sizes
64     empty_img = np.zeros((min(height, limit), min(width, limit), 3), dtype=np
.uint8)
65     cprint(f"Created a blank canvas with dimensions: {empty_img.shape}", "
green")
66     return empty_img
67
68 # Function to warp an image back using a homography matrix

```

```

69 def apply_p2p_warping(img: np.ndarray, homography_matrix: np.ndarray) -> np.
   ndarray:
70     # Calculate the corner points of the original image
71     img_corners = np.array([[0, 0], [0, img.shape[0]], [img.shape[1], img.
       shape[0]], [img.shape[1], 0]])
72     img_corners_homo = np.append(img_corners, np.ones((4, 1)), axis=1)
73     # Apply the inverse homography matrix to the corners to find new
       coordinates
74     transformed_corners = np.linalg.inv(homography_matrix).dot(
       img_corners_homo.T)
75     transformed_corners /= transformed_corners[-1] # Normalize
76     transformed_corners = transformed_corners.astype(int)
77     # Find the offset and new dimensions for the resulting image
78     x_offset = min(transformed_corners[0])
79     y_offset = min(transformed_corners[1])
80     new_img_width = max(transformed_corners[0]) - min(transformed_corners[0])
81     new_img_height = max(transformed_corners[1]) - min(transformed_corners
       [1])
82     cprint(f"New image dimensions: Width={new_img_width}, Height={
       new_img_height}", "yellow")
83     # Create a blank image with the calculated size
84     output_img = create_empty_canvas(new_img_width, new_img_height)
85     # Iterate through each pixel in the new image and map it back to the
       original
86     for col in range(output_img.shape[1]):
87         for row in range(output_img.shape[0]):
88             x_coord = col + x_offset
89             y_coord = row + y_offset
90
91             # Apply the homography to map the pixel back to the original
       image coordinates
92             projected_point = homography_matrix.dot([x_coord, y_coord, 1])
93             original_x = round(projected_point[0] / projected_point[2])
94             original_y = round(projected_point[1] / projected_point[2])
95
96             # Check if the calculated coordinates are within bounds of the
       original image
97             if 0 <= original_x < img.shape[1] and 0 <= original_y < img.shape
       [0]:
98                 output_img[row, col] = img[original_y, original_x]
99
100     cprint("Image transformation completed using inverse homography.", "green
       ")
101     return output_img
102
103 def Task1_p2p(domain_pts: np.ndarray, ratio: int, img, image_name: str,
   task_number: str):
104     print(image_name)
105     range_points = get_range_points(domain_pts, ratio)
106     distorted_image = np.copy(img)
107     H_inverse = np.linalg.inv(calc_point_to_point_homography(domain_pts,
       range_points))
108     result_img = apply_p2p_warping(distorted_image, H_inverse)
109     cv2.imwrite(f'Final_Results/Task{task_number}_P2P_{image_name}.jpg',
       result_img)
110     '''-----END of Task 1 Point
       to Point-----'''
111

```

```

112 '''-----Task 1 Two Step
113 -----'''
114 def calculate_projective_homography(points: np.ndarray) -> np.ndarray:
115     # Extract points P, Q, R, S
116     P, Q, R, S = points
117     # Compute lines between points
118     line_pq, line_rs, line_pr, line_qs, = np.cross(P, Q) , np.cross(R, S) ,
119     np.cross(P, R) , np.cross(Q, S)
120     # Find the intersection points of the lines and cast to float64
121     intersection_pq_rs = np.cross(line_pq, line_rs).astype(np.float64)
122     intersection_pr_qs = np.cross(line_pr, line_qs).astype(np.float64)
123     # Normalize the intersection points (convert from homogeneous to
124     # Euclidean coordinates)
125     if intersection_pq_rs[2] != 0:
126         intersection_pq_rs /= intersection_pq_rs[2]
127     if intersection_pr_qs[2] != 0:
128         intersection_pr_qs /= intersection_pr_qs[2]
129     # Compute the vanishing line by taking the cross product of the two
130     # intersection points
131     vanishing_line = np.cross(intersection_pq_rs, intersection_pr_qs).astype(
132         np.float64)
133     # Normalize the vanishing line
134     if vanishing_line[2] != 0:
135         vanishing_line /= vanishing_line[2]
136     # Construct the projective homography matrix H
137     projective_homography = np.eye(3, dtype=np.float64)
138     projective_homography[2] = vanishing_line # Set the vanishing line in
139     # the last row
140     return projective_homography
141
142 def apply_two_step_warping(H: np.ndarray, x_min: int, y_min: int, img1: np.
143     ndarray, distorted_image: np.ndarray) -> np.ndarray:
144     undistorted_image = np.copy(distorted_image)
145     for i in range(distorted_image.shape[1]):
146         for j in range(distorted_image.shape[0]):
147             x, y, w = np.dot(np.linalg.inv(H), np.array([i+x_min, j+y_min,
148                 1]))
149             x_coord = int(np.floor(x/w))
150             y_coord = int(np.floor(y/w))
151             if y_coord > 0 and y_coord < img1.shape[0] and x_coord > 0 and
152                 x_coord < img1.shape[1]:
153                 undistorted_image[j,i] = img1[y_coord, x_coord]
154     return undistorted_image
155
156 def calculate_affine_homography(points: np.ndarray) -> np.ndarray:
157     # Extract points for calculation of lines
158     P, Q, R, S = points
159     # Calculate the lines connecting points
160     line_pq = np.cross(P, Q)
161     line_pr = np.cross(P, R)
162     line_qs = np.cross(Q, S)
163     line_rs = np.cross(R, S)
164     # Normalize the lines
165     line_pq_normalized = line_pq / line_pq[2]
166     line_pr_normalized = line_pr / line_pr[2]
167     line_qs_normalized = line_qs / line_qs[2]
168     line_rs_normalized = line_rs / line_rs[2]
169     # Create matrix A using the line equations
170     A = np.array([

```

```

162         [line_pq_normalized[0] * line_pr_normalized[0],
163           line_pq_normalized[0] * line_pr_normalized[1] + line_pq_normalized
164             [1] * line_pr_normalized[0]],
165         [line_qs_normalized[0] * line_rs_normalized[0],
166           line_qs_normalized[0] * line_rs_normalized[1] + line_qs_normalized
167             [1] * line_rs_normalized[0]]
168     ])
169     # Create vector b using the normalized lines
170     b = np.array([
171         -line_pq_normalized[1] * line_pr_normalized[1],
172         -line_qs_normalized[1] * line_rs_normalized[1]
173     ])
174     # Solve for s
175     try:
176         s = np.linalg.solve(A, b)
177     except np.linalg.LinAlgError:
178         cprint("Matrix A is singular, using pseudo-inverse.", "yellow")
179         s = np.linalg.pinv(A) @ b
180     # Construct the symmetric matrix S
181     S = np.eye(2) # Initialize a 2x2 identity matrix
182     S[0, 0] = s[0]
183     S[0, 1] = s[1]
184     S[1, 0] = s[1]
185     S[1, 1] = s[1]
186     # Decompose S to obtain the affine transformation matrix
187     U, D, V = np.linalg.svd(S)
188     sqrt_D = np.sqrt(np.diag(D))
189     transformation_matrix = U @ sqrt_D @ U.T
190     # Construct the 3x3 homography matrix
191     affine_homography = np.eye(3)
192     affine_homography[0:2, 0:2] = transformation_matrix
193     return affine_homography
194
195 def Task1_TwoStep(domain_pts: np.ndarray, ratio: int, img, image_name: str,
196 task_number: str):
197     range_pts = get_range_points(domain_pts, ratio)
198
199     # REMOVING Projective:
200     distortion_removed = "projection"
201     H_vp_inverse = np.linalg.inv(calculate_projective_homography(domain_pts))
202     # H_vp_inverse /= H_vp_inverse[2,2]
203     x_min, x_max, y_min, y_max = calc_bounds(H_vp_inverse, domain_pts)
204     distorted_image = np.zeros((y_max-y_min+1, x_max-x_min+1,3),np.uint8)
205     result_img = apply_two_step_warping(H_vp_inverse, x_min, y_min, np.copy(
206         img), distorted_image)
207     cv2.imwrite(f"Final_Results/Task{task_number}_TwoStep_{distortion_removed}
208         _{image_name}.jpg",result_img)
209     cprint("Removed Projectie Distortion", "green")
210
211     # REMOVING AFFINE:
212     distortion_removed = "affine"
213     H_affine = np.linalg.inv(calculate_affine_homography(domain_pts ))
214     H_affine /= H_affine[2,2]
215     x_min, x_max, y_min, y_max = calc_bounds(H_affine, domain_pts)
216     distorted_image = np.zeros((y_max-y_min+1, x_max-x_min+1,3),np.uint8)
217     result_img = apply_two_step_warping(H_affine, x_min, y_min, np.copy(img),
218         distorted_image)
219     cv2.imwrite(f"Final_Results/Task{task_number}_TwoStep_{distortion_removed}
220         _{image_name}.jpg",result_img)
221     cprint("Removed Affine Distortion", "green")

```

```

214
215     # # REMOVING Both Distortions:
216     distortion_removed = "both"
217     H_overall = H_vp_inverse @ H_affine
218     H_overall /= H_overall[2,2]
219     x_min, x_max, y_min, y_max = calc_bounds(H_overall, domain_pts)
220     distorted_image = np.zeros((y_max-y_min+1, x_max-x_min+1,3),np.uint8)
221     result_img = apply_two_step_warping(H_overall, x_min, y_min, np.copy(img)
222         , distorted_image)
222     cv2.imwrite(f"Final_Results/Task{task_number}_TwoStep_{distortion_removed}
223         _{image_name}.jpg",result_img)
223     cprint("Removed Both Projectie & Affine Distortion", "green")
224
225     '''-----END of Task 1 Two
226     Step-----'''
227
227     '''-----Task 1 One Step
228     -----'''
228     # Breaks sometimes, need to account for float int type of points
229     def calc_lines(domain_pts: np.ndarray) -> List[np.ndarray]:
230         def line_bw_points(p1, p2):
231             p1_coord = np.array([p1[0],p1[1],1])
232             p2_coord = np.array([p2[0],p2[1],1])
233             l = np.cross(p1_coord, p2_coord).astype(float)
234             l /= np.linalg.norm(l)
235             return l
236         if len(domain_pts) > 4:
237             P, Q, R, S, _, _, _ = domain_pts
238         else:
239             P, Q, R, S = domain_pts
240         return [line_bw_points(P,Q) , line_bw_points(P,S) ,line_bw_points(R,S),
241             line_bw_points(Q,R), line_bw_points(Q,S), line_bw_points(P,R)]
241
242     def compute_homography_onestep_affine(pts: np.ndarray) -> np.ndarray:
243         # Calculate 5 pairs of orthogonal lines
244         l1 = np.cross(pts[0], pts[1]) # PQ
245         m1 = np.cross(pts[1], pts[3]) # QS
246
247         l2 = np.cross(pts[1], pts[3]) # QS
248         m2 = np.cross(pts[3], pts[2]) # SR
249
250         l3 = np.cross(pts[3], pts[2]) # SR
251         m3 = np.cross(pts[2], pts[0]) # RP
252
253         l4 = np.cross(pts[2], pts[0]) # RP
254         m4 = np.cross(pts[0], pts[1]) # PQ
255
256         l5 = np.cross(pts[0], pts[3]) # PS
257         m5 = np.cross(pts[2], pts[1]) # RQ
258
259         # Build the X matrix and y vector for least square estimation
260         def build_x(l: np.ndarray, m: np.ndarray) -> np.ndarray:
261             return np.array([
262                 l[0] * m[0],
263                 (l[1] * m[0] + l[0] * m[1]) / 2,
264                 l[1] * m[1],
265                 (l[2] * m[0] + l[0] * m[2]) / 2,
266                 (l[2] * m[1] + l[1] * m[2]) / 2
267             ])

```

```

268 X1 = build_x(l1, m1)
269 X2 = build_x(l2, m2)
270 X3 = build_x(l3, m3)
271 X4 = build_x(l4, m4)
272 X5 = build_x(l5, m5)
273
274
275 X = np.vstack((X1, X2, X3, X4, X5))
276 y = np.array([-m1[2]*l1[2], -m2[2]*l2[2], -m3[2]*l3[2], -m4[2]*l4[2], -m5
277               [2]*l5[2]], dtype=float)
278
279 # Solve for the dual degenerate conic parameters
280 param = np.linalg.lstsq(X, y, rcond=None)[0]
281
282 # Construct the image of the dual degenerate conic (dual_degenerate_conic
283 )
284 dual_degenerate_conic = np.zeros((3, 3))
285 dual_degenerate_conic[0, :] = [param[0], param[1] / 2, param[3] / 2]
286 dual_degenerate_conic[1, :] = [param[1] / 2, param[2], param[4] / 2]
287 dual_degenerate_conic[2, :] = [param[3] / 2, param[4] / 2, 1]
288 dual_degenerate_conic /= np.max(dual_degenerate_conic)
289
290 # Decompose the conic to find A and v for the homography matrix
291 S = dual_degenerate_conic[:2, :2]
292 U, D, Vt = np.linalg.svd(S)
293 A = U @ np.sqrt(np.diag(D)) @ Vt
294 v = np.linalg.inv(A.T) @ dual_degenerate_conic[2, :2]
295
296 # Construct the homography matrix H
297 H = np.eye(3)
298 H[:2, :2] = A
299 H[2, :2] = v
300
301 return H
302
303 # Function to warp an image back using a homography matrix and apply correct
304 scaling
305 def apply_p2p_warping_with_scaling(img: np.ndarray, homography_matrix: np.
306 ndarray) -> np.ndarray:
307     # Calculate the corner points of the original image
308     img_corners = np.array([[0, 0], [0, img.shape[0]], [img.shape[1], img.
309 shape[0]], [img.shape[1], 0]])
310     img_corners_homo = np.append(img_corners, np.ones((4, 1)), axis=1)
311     # Apply the inverse homography matrix to the corners to find new
312     coordinates
313     transformed_corners = np.linalg.inv(homography_matrix).dot(
314         img_corners_homo.T)
315     transformed_corners /= transformed_corners[-1] # Normalize to
316     homogeneous coordinates
317     transformed_corners = transformed_corners.astype(int)
318     # Find the offset and new dimensions for the resulting image
319     x_offset = min(transformed_corners[0])
320     y_offset = min(transformed_corners[1])
321     new_img_width = max(transformed_corners[0]) - min(transformed_corners[0])
322     new_img_height = max(transformed_corners[1]) - min(transformed_corners
323 [1])
324     # Ensure scaling is correct by keeping the aspect ratio
325     scaling_factor_x = img.shape[1] / new_img_width
326     scaling_factor_y = img.shape[0] / new_img_height

```

```

318 scaling_factor = min(scaling_factor_x, scaling_factor_y)
319 new_img_width = int(new_img_width * scaling_factor)
320 new_img_height = int(new_img_height * scaling_factor)
321 # Create a blank image with the calculated size
322 output_img = np.zeros((new_img_height, new_img_width, 3), dtype=np.uint8)
323 # Iterate through each pixel in the new image and map it back to the
    original
324 for col in range(output_img.shape[1]):
325     for row in range(output_img.shape[0]):
326         # Apply scaling factor to keep the aspect ratio
327         x_coord = (col + x_offset) / scaling_factor
328         y_coord = (row + y_offset) / scaling_factor
329         # Apply the homography to map the pixel back to the original
            image coordinates
330         projected_point = homography_matrix.dot([x_coord, y_coord, 1])
331         original_x = round(projected_point[0] / projected_point[2])
332         original_y = round(projected_point[1] / projected_point[2])
333         # Check if the calculated coordinates are within bounds of the
            original image
334         if 0 <= original_x < img.shape[1] and 0 <= original_y < img.shape
            [0]:
335             output_img[row, col] = img[original_y, original_x]
336     return output_img
337
338 def Task1_OneStep(domain_pts: np.ndarray, ratio: int, img, image_name: str,
task_number: str):
339     H_one = compute_homography_onestep_affine(domain_pts)
340     H_one_inv = np.linalg.inv(H_one)
341     distorted = np.copy(img)
342     # if image_name == "corridor_1":
343     #     H_one_inv = np.linalg.inv(calculate_affine_homography(domain_pts
        [0:4]))
344     result_img = apply_p2p_warping_with_scaling(np.copy(img), H_one_inv)
345     cv2.imwrite(f"Final_Results/TEST_Task{task_number}_OneStep_{image_name}.
        jpg", result_img)
346 '''-----END of Task 1 One
    Step-----'''
347
348 # Main execution block
349 if __name__ == "__main__":
350     ##### TASK 1 P2P
        #####
351
352     ##### Declaration Variables
        #####
353     # Load the distorted board image
354     img_path = 'HW3_images/board_1.jpeg'
355     img = io.imread(img_path)
356     img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
357     board_image_points = np.array([[72, 411, 1], [1220, 141, 1], [1350, 1955,
        1], [419, 1755, 1]])
358     ratio = 12/8
359     # Task1_p2p(board_image_points, ratio, img_bgr, "board_1", 1)
360
361     img_path = 'HW3_images/corridor.jpeg'
362     img = io.imread(img_path)
363     img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
364     corridor_image_points = np.array([[1087, 534, 1 ], [1307, 493, 1 ],
        [1298, 1344, 1 ], [1081, 1223, 1 ]])

```

```

365 ratio = 6/3
366 # Task1_p2p(corridor_image_points, ratio, img_bgr, "corridor_1", 1)
367 cprint("Transformation complete and saved as task1_p2p.jpg.", "green")
368 ##### End of TASK 1 P2P
369 #####
370
371 ##### TASK 1 Two - Step
372 #####
373 # Load the distorted board image
374 img_path = 'HW3_images/board_1.jpeg'
375 img = io.imread(img_path)
376 img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
377 board_image_points = np.array([[72, 411, 1], [1220, 141, 1], [1350, 1955,
378 1], [419, 1755, 1]])
379 ratio = 12/8
380 # Task1_TwoStep(board_image_points, ratio, img_bgr, "board_1", 1)
381
382 img_path = 'HW3_images/corridor.jpeg'
383 img = io.imread(img_path)
384 img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
385 corridor_image_points = np.array([[1087, 534, 1 ], [1307, 493, 1 ],
386 [1298, 1344, 1 ], [1081, 1223, 1 ]])
387
388 ratio = 6/3
389 # Task1_TwoStep(corridor_image_points, ratio, img_bgr, "corridor_1", 1)
390 cprint("Transformation complete and saved as task1_TwoStep.jpg.", "green")
391 ##### End of TASK 1 Two - Step
392 #####
393
394 ##### TASK 1 One - Step
395 #####
396 # Load the distorted board image
397 img_path = 'HW3_images/board_1.jpeg'
398 img = io.imread(img_path)
399 img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
400 board_image_points = np.array([[72, 411, 1], [1220, 141, 1], [1350, 1955,
401 1], [419, 1755, 1], \
402 [878, 539, 1], [897, 653, 1], [601, 693, 1], \
403 [852, 305, 1], [865, 423, 1], [761, 444, 1]
404 ])
405 ratio = 12/8
406 # Task1_OneStep(board_image_points, ratio, img_bgr, "board_1", 1)
407
408 img_path = 'HW3_images/corridor.jpeg'
409 img = io.imread(img_path)
410 img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
411 corridor_image_points = np.array([[1087, 534, 1 ], [1307, 493, 1 ],
412 [1298, 1344, 1 ], [1081, 1223, 1 ], \
413 [1122, 856, 1], [1119, 1207, 1], [1205, 1252, 1], \
414 [1117, 518, 1], [1183, 1268, 1], [1093, 1204, 1]
415 ])
416 ratio = 6/3
417
418 point_display(img, corridor_image_points)
419
420 Task1_OneStep(corridor_image_points, ratio, img_bgr, "corridor_1", 1)
421 cprint("Transformation complete and saved as task1_OneStep.jpg.", "green")

```

```

415 ##### End of TASK 1 One - Step
416 #####
417 ##### TASK 2 MY Images:
418 #####
418 cprint("Testing My Images:", "white")
419 # My first image info:
420 img_path1 = 'MyImages/keyboard.jpeg'
421 img1 = io.imread(img_path1)
422 img_bgr1 = cv2.cvtColor(img1, cv2.COLOR_RGB2BGR)
423 ratio1 = 0.08333333*(13/6) # These are in inches being converted to feet
424
425 keyboard_points = np.array([[1029, 67, 1], [1090, 1394, 1], [520, 1422,
426 1], [424, 132, 1]])
427 point_display(img1, keyboard_points)
428
428 # Run for each task:
429 # Task1_p2p(keyboard_points, ratio1, img_bgr1, "keyboard", 2)
430 # Task1_TwoStep(keyboard_points, ratio1, img_bgr1, "keyboard", 2)
431 # Task1_OneStep(keyboard_points, ratio1, img_bgr1, "keyboard", 2)
432
433 # keyboard_points = np.array([1029, 67, 1], [1090, 1394, 1], [520, 1422,
434 1], [424, 132, 1])
435 cprint("Successfully completed run for keyboard image!", "green")
436
437
438 # _=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_=_
439 # My second image
440 img_path2 = 'MyImages/door.jpeg'
441 img2 = io.imread(img_path2)
442 img_bgr2 = cv2.cvtColor(img2, cv2.COLOR_RGB2BGR)
443 ratio2 = 0.08333333*(25/9)
444
445 door_points = np.array([[813, 465, 1], [1019, 466, 1], [1001, 987, 1],
446 838, 992, 1]])
447 point_display(img2, door_points)
448
449
450
451 Task1_p2p(door_points, ratio2, img_bgr2, "door", 2)
452 Task1_TwoStep(door_points, ratio2, img_bgr2, "door", 2)
453 Task1_OneStep(door_points, ratio2, img_bgr2, "door", 2)
454
455 cprint("Successfully completed run for door image!", "green")
456
457 # cprint("Transformation complete and saved as Task2_OneStep.jpg.", "green")
458
459 ##### TASK 2:
460 #####

```

Listing 1: Python code for Task-1 and Task-2

References